Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т
по предмету Методы оптимизации
Тема задания: «Лабораторная №2. Метод ломаных»

Обучающийся: Левахин Лев Александрович, 343730, K3340
Вариант: 13

Санкт-Петербург,
2026

Введение

В этой лабораторной нужно было написать программу, которая находит глобальный экстремум (минимум или максимум) для функции одной переменной на заданном отрезке. Метод — **метод ломаных** (метод Пиявского). Суть в том, что функция обязана быть *липшицевой*, то есть у неё есть такая константа L, что график не может уйти вниз от известной точки быстрее, чем со скоростью L. Благодаря этому мы можем строить конусы от точек испытаний и получать нижнюю оценку (ломаная), которая всегда лежит под графиком

Было сделано:

- парсер строки — чтобы можно было вбить функцию руками, например `x + sin(3.14159*x)`;
- визуализацию: график функции, ломаная, точки испытаний, сама найденная точка экстремума;
- вывод аргумента, значения функции, числа итераций и времени работы;
- поиск как минимума, так и максимума (через поиск минимума от $-f$).

0.1 Что такое липшицева функция

Функция $f(x)$ называется липшицевой на отрезке $[a, b]$, если существует такая константа $L \geq 0$, что для любых двух точек x_1, x_2 выполняется:

$$|f(x_1) - f(x_2)| \leq L\,|x_1 - x_2|.$$

Простыми словами: график не может прыгнуть резко вверх или вниз - наклон ограничаем числом L. Если мы знаем значение функции в точке x_i, то в любой другой точке график не может опуститься ниже, чем на $L \cdot |x - x_i|$. Это и есть так называемый конус:

$$\varphi_i(x) = f(x_i) - L\,|x - x_i|.$$

0.2 Как устроена ломаная

Если у нас есть несколько испытаний $x_1, x_2, \ldots, x_n$, то в каждой точке график функции лежит выше каждого из конусов. Значит, он лежит выше их максимума:

$$\varphi(x) = \max_i \varphi_i(x).$$

Эта функция $\varphi(x)$ и есть **ломаная** (нижняя огибающая конусов). Она всегда лежит **под** графиком $f(x)$ и даёт нам нижнюю оценку минимума

0.3 Где делать новое испытание

Самое «интересное» место — там, где ломаная выше всего поднимается, то есть в её **вершине**. Там мы ещё ничего не знаем о функции, но из-за липшицевости там возможен минимум. Если строить ломаную через вершины (точки пересечения конусов), то она будет быстро «прижиматься» к графику.

Важно: вершина считается как точка пересечения двух соседних конусов. Для двух испытаний (x_1, y_1) и (x_2, y_2), где $x_1 < x_2$:

$$y_1 - L(x - x_1) = y_2 - L(x_2 - x),$$

откуда

$$x = \frac{y_1 - y_2 + L\,(x_1 + x_2)}{2L}.$$

Именно по этой формуле в коде ищется новая точка. Это принципиально отличает метод от «мелкого разбиения» (которое в задании штрафуется).

0.4 Критерий остановки

Мы знаем две оценки истинного минимума f^*:

- **сверху**: лучшее найденное значение $f_{best} = \min_i f(x_i)$ (минимум функции не может быть больше того, что мы уже нашли);
- **снизу**: максимум ломаной $\max \varphi(x)$ (минимум не может быть ниже ломаной, потому что она под графиком).

Значит, $f^* \in [\max \varphi,\ \ f_{best}]$. Если разброс этого интервала меньше ε, то минимум найден с точностью ε:

$$f_{best} - \max \varphi \le \varepsilon.$$

Это и есть критерий остановки

0.5 Поиск максимума

Если надо максимум — ищем минимум функции $-f(x)$ и меняем знак у найденного значения. Метод ломаных по построению ищет именно минимум.

1 Реализация программы

1.1 Парсер функции из строки

Сначала я написал парсер строки, чтобы можно было при желании вбивать функцию через консоль от руки. Например, пользователь вводит `x + sin(3.14159*x)`, а наш словарь это переводит в нампи функции

```
SAFE_NAMESPACE = {
    "x": None,
    "sin": np.sin, "cos": np.cos, "tan": np.tan,
    "exp": np.exp, "log": np.log, "sqrt": np.sqrt,
    "abs": np.abs, "pi": np.pi, "e": np.e,
    "arcsin": np.arcsin, "arccos": np.arccos, "arctan": np.arctan,
    "sinh": np.sinh, "cosh": np.cosh, "tanh": np.tanh,
}

def make_function(expr: str) -> Callable[[float], float]:
    code = compile(expr, "<func>", "eval")
    def f(x):
        env = dict(SAFE_NAMESPACE)
        env["x"] = x
        return eval(code, {"__builtins__": {}}, env)
    return f
```

Мы ограничили набор имён только математикой и запретили встроенные функции через `"__builtins__": {}`

1.2 Оценка константы Липшица

Константу L по-хорошему должен давать пользователь, но если он не знает, можно её оценить численно как максимум производной на мелкой сетке (с небольшим запасом 10%):

```
def estimate_lipschitz(f, a, b, n=1000):
    xs = np.linspace(a, b, n)
    ys = f(xs)
    slopes = np.abs(np.diff(ys) / np.diff(xs))
    L = float(np.max(slopes))
    return L * 1.1 if L > 0 else 1.0
```

Запас 10% нужен, чтобы L заведомо была не меньше настоящей: условие Липшица тогда гарантированно выполняется, и ломаная точно лежит под графиком. Если бы L оказалась меньше настоящей, конусы могли бы сломать график

1.3 Формула вершины ломаной

```
def _vertex(x1, y1, x2, y2, L):
    return (y1 - y2 + L * (x1 + x2)) / (2.0 * L)
```

Это та формула, что выведена в теории:

$$x = \frac{y_1 - y_2 + L\,(x_1 + x_2)}{2L}.$$

Это и есть та самая точка пересечения конусов, а не середина отрезка

1.4 Значение ломаной в произвольной точке

Чтобы понять, насколько высоко поднялась ломаная в данной точке x, нужно взять максимум по всем конусам:

```
def _broken_value_at(xv, points, L):
    best = -np.inf
    for xi, yi in points:
        v = yi - L * abs(xv - xi)
        if v > best:
            best = v
    return best
```

Мы проверяем **все** испытания, а не только двух соседей

1.5 Основной цикл метода

```
def broken_lines(f, a, b, eps, L=None, maximize=False, max_iter=100_000):
    t0 = time.perf_counter()
    g = (lambda x: -f(x)) if maximize else f
    if L is None:
        L = estimate_lipschitz(g, a, b)
    points = [(a, float(g(a))), (b, float(g(b)))]

    converged = False
    for it in range(1, max_iter + 1):
        points.sort(key=lambda p: p[0])

        #
        best_R = -np.inf
        best_x = None
        for i in range(len(points) - 1):
            x1, y1 = points[i]
            x2, y2 = points[i + 1]
            if x2 - x1 < 1e-15:
                continue
            xv = _vertex(x1, y1, x2, y2, L)
            if xv <= x1 or xv >= x2:
                xv = 0.5 * (x1 + x2)
            R = _broken_value_at(xv, points, L)
            if R > best_R:
                best_R = R
                best_x = xv

        #                                    :

        f_best = min(y for _, y in points)
        if best_x is None or (f_best - best_R) <= eps:
            converged = (best_x is not None) and ((f_best - best_R) <= eps
    )
            break

        #

        if any(abs(best_x - px) < 1e-12 for px, _ in points):
```

```
            for i in range(len(points) - 1):
                x1, _ = points[i]
                x2, _ = points[i + 1]
                if x1 <= best_x <= x2 and (x2 - x1) > 1e-12:
                    best_x = 0.5 * (x1 + x2)
                    break
        y_new = float(g(best_x))
        points.append((best_x, y_new))

    elapsed = time.perf_counter() - t0
    best_x, best_y = min(points, key=lambda p: p[1])
    f_val = -best_y if maximize else best_y
    return Result(x_min=best_x, f_min=f_val,
                  iterations=len(points) - 2,
                  elapsed=elapsed, history=points,
                  L=L, converged=converged)
```

- На старте только две точки — концы отрезка a и b. Дальше метод сам решает, куда бить
- На каждой итерации пересчитываем **все** вершины ломаной между соседними испытаниями и выбираем **самую высокую** — там минимум неопределён сильнее всего
- Критерий остановки — разброс между верхней оценкой (`f_best`) и нижней (`best_R`)
- Защита от зацикливания: если новая точка почти совпала с существующей, сдвигаем её в середину интервала. Иначе программа может застрять

1.6 Визуализация

```
def plot_result(f, a, b, res, title="", maximize=False):
    g_plot = (lambda x: -f(x)) if maximize else f
    L = res.L
    pts = sorted(res.history, key=lambda p: p[0])

    xs_plot = np.linspace(a, b, 800)
    lower = np.full_like(xs_plot, -np.inf)
    for xi, yi in pts:
        cone = yi - L * np.abs(xs_plot - xi)
        lower = np.maximum(lower, cone)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(xs_plot, g_plot(xs_plot), 'b-', label='f(x)', linewidth=1.8)
    ax.plot(xs_plot, lower, 'r--', label='                ', linewidth=1.2)
    ax.plot([p[0] for p in pts], [p[1] for p in pts],
            'ko', markersize=4, label='                  ')
    ax.plot(res.x_min, res.f_min, 'g*', markersize=16,
            label=f'                    ({res.x_min:.6f}, {res.f_min:.6f})'
    )
    ax.set_xlabel('x'); ax.set_ylabel('f(x)')
    ax.set_title(title)
```

```
    ax.legend(); ax.grid(True, alpha=0.3)
    plt.tight_layout(); plt.show()
```

Ломаная строится на плотной сетке — это **только для картинки**. Сами испытания всё равно кладутся в **вершины**, а не на сетку

2 Тестовые функции

По условию нужно, чтобы на отрезке было **не менее двух локальных минимумов**. Я взял четыре функции:

1. $f(x) = x + \sin(\pi x)$ на $[-2, 2]$

Две локальных «ямки» внутри, плюс глобальный минимум лежит на **границе** $x = -2$, где $f = -2$

2. Функция Растригина (1D)

$$f(x) = x^2 - 10\cos(2\pi x) + 10.$$

Волнистая функция с кучей локальных минимумов. Глобальный минимум один — в $x = 0$, $f(0) = 0$. На $[-5.12, 5.12]$ тут около 20 локальных минимумов

3. Функция Экли (1D, упрощённая)

$$f(x) = -20e^{-0.2|x|} - e^{\cos(2\pi x)} + e + 20.$$

Тоже несколько локальных минимумов, глобальный - в $x = 0$, $f(0) = 0$

4. $f(x) = x\sin x + x^2/20$ на $[-10, 10]$

Волнистая функция с ростом на краях. Есть несколько локальных минимумов и максимумов — удобно тестировать и поиск минимума, и поиск максимума

3 Результаты работы программы

Во всех тестах использовано $\varepsilon = 10^{-3}$

3.1 Пример 1: $f(x) = x + \sin(\pi x)$

```
L (оценка)  : 4.5557
x*          : -2.00000000
f(x*)       : -2.00000000
итераций    : 5
сошлось     : True
время       : 0.18 мс
```

Глобальный минимум действительно в $x = -2$ на границе, $f = -2$

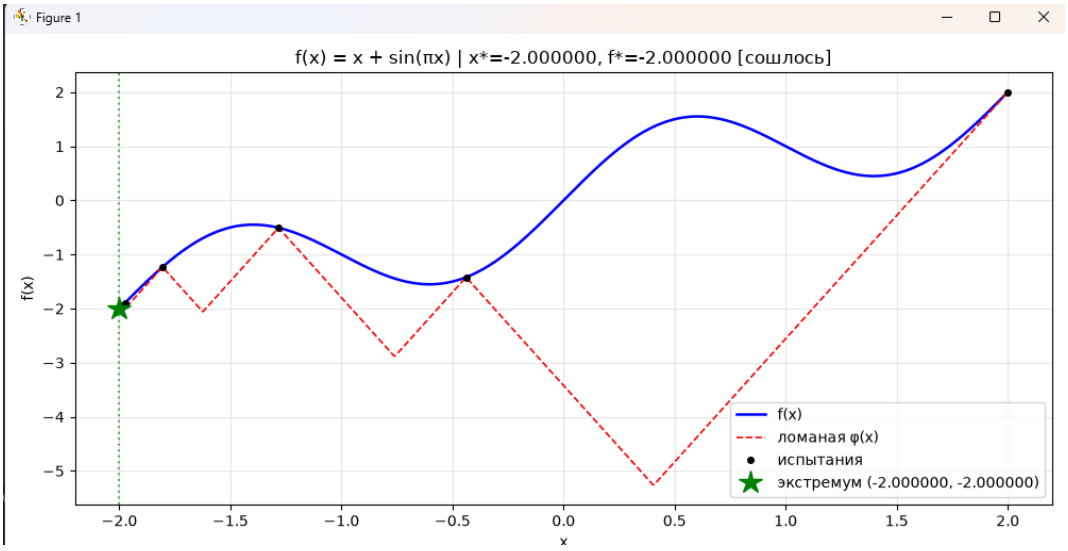


Рис. 1: Метод ломаных для $f(x) = x + \sin(\pi x)$

3.2 Пример 2: 1D Растригин

```
L (оценка)  : 78.4413
x*          : 0.00000000
f(x*)       : 0.00000000
итераций    : 6
сошлось     : True
время       : 0.25 мс
```

$f(0) = 0$, это глобальный минимум

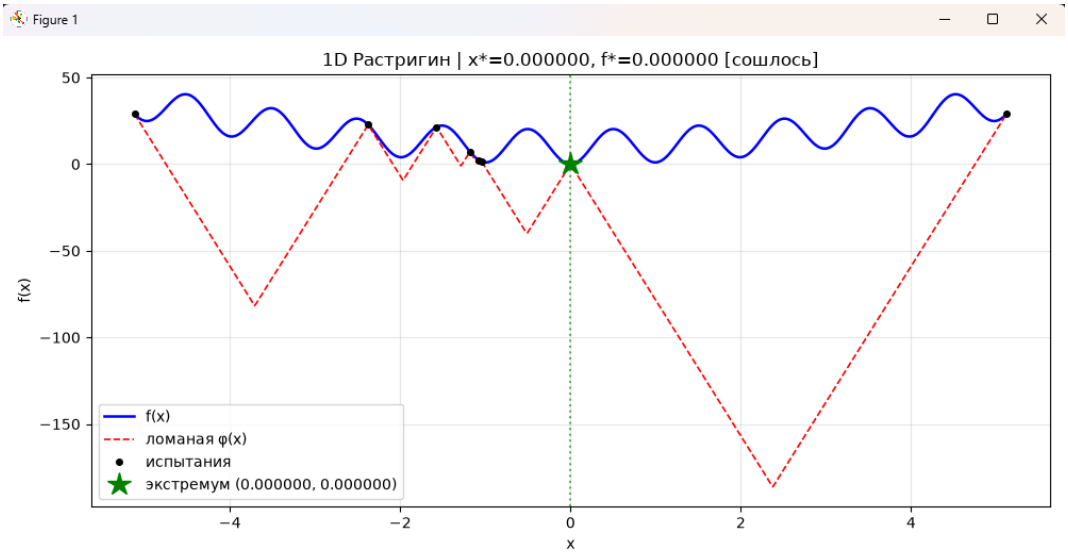


Рис. 2: Метод ломаных для 1D-функции Растригина на $[-5.12, 5.12]$. Глобальный минимум: $x^* = 0$, $f^* = 0$.

3.3 Пример 3: 1D Экли

```
L (оценка)  : 14.3550
x*          : 0.00000000
f(x*)       : 0.00000000
итераций    : 3
сошлось     : True
время       : 0.40 мс
```

$f(0) = 0$.

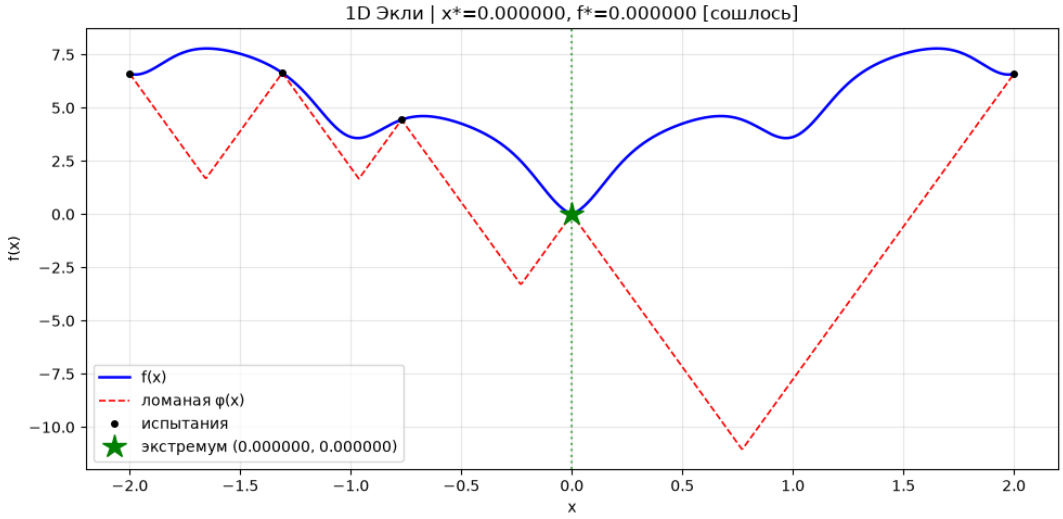


Рис. 3: Метод ломаных для 1D-функции Экли на $[-2, 2]$. Глобальный минимум $x^* = 0$, $f^* = 0$.

3.4 Пример 4: $f(x) = x \sin x + x^2/20$ (поиск минимума)

```
L (оценка)  : 9.5354
x*          : -5.02308289
f(x*)       : -3.52101792
итераций    : 5
сошлось     : True
время       : 0.26 мс
```

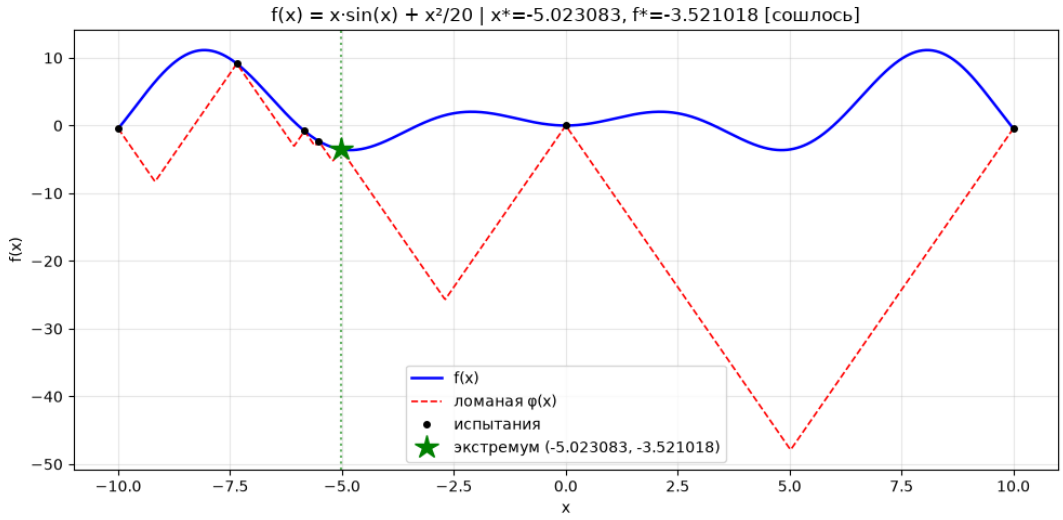


Рис. 4: Метод ломаных для $f(x) = x \sin x + x^2/20$ на $[-10, 10]$ (поиск минимума). Найден $x^* \approx -5.023$, $f^* \approx -3.521$.

3.5 Пример 5: та же функция, но поиск максимума

```
L (оценка)  : 9.5354
x*          : -8.25642672
f(x*)       : 11.00521754
итераций    : 8
сошлось     : True
время       : 0.31 мс
```

Здесь видно, что для поиска максимума делается чуть больше итераций (8 против 5) - как я понимаю это потому что ломаная строится вокруг другого места. Всё потому, что для поиска максимума мы минимизируем $-f$, а функциия немного другая

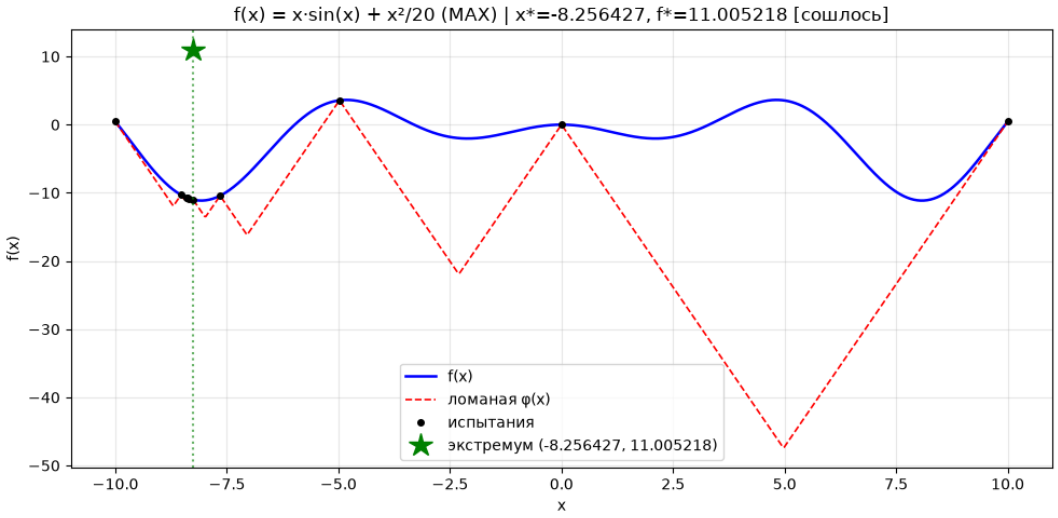


Рис. 5: Метод ломаных для $f(x) = x\sin x + x^2/20$ на $[-10, 10]$ (поиск максимума через минимизацию $-f$). Найден $x^* \approx -8.256$, $f^* \approx 11.005$.

3.6 Пример 6: функция, введённая строкой

Пользователь вводит `"x + sin(3.14159*x)"` — программа парсит строку и потом решает задачу

```
L (оценка)  : 4.5557
x*          : -2.00000000
f(x*)       : -1.99999469
итераций    : 4
сошлось     : True
время       : 0.21 мс
```

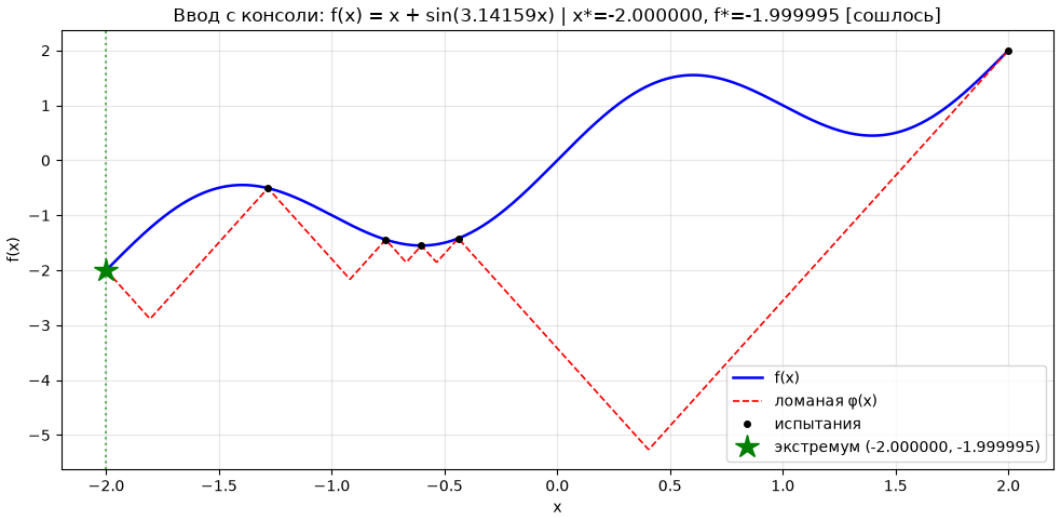


Рис. 6: Метод ломаных для функции, введённой пользователем строкой: $f(x) = x + \sin(3.14159x)$ на $[-2, 2]$. Парсер безопасно превращает строку в функцию

4 Выводы

- Метод ломаных действительно быстро находит **глобальный** экстремум липшицевой функции без перебора сетки
- На всех тестовых функциях программа нашла правильный ответ: Растригин $x = 0$, Экли $x = 0$, $x + \sin(\pi x)$ — граничный минимум -2, и т.д.
- Критерий остановки честный — разброс между верхней и нижней оценками меньше ε
- Программа принимает функцию строкой, рисует всё нужное (график, ломаную, точки), выводит все параметры
- Поиск максимума работает через поиск минимума $-f$